# PaperToSkill: Turning Research Papers into Portable Agent Skills

**Anonymous submission**


## Abstract

Many LLM and agent papers introduce workflows that could help ordinary users and agent builders, but those workflows are usually described for human reading rather than direct reuse. Summaries compress the idea, but often discard the procedural details, validation checks, assumptions, and failure branches needed to apply the method. We introduce PaperToSkill, a paper-to-skill conversion workflow that turns source-anchored paper notes into compact, human-editable agent skills. Each skill records the paper's contribution, use conditions, workflow, validation checks, failure cases, transfer notes, and source anchors. In a four-paper benchmark over AI Scientist-v2, Reflexion, AIDE, and Toolformer, PaperToSkill generates skills that score 20/20 on deterministic structural rubrics, preserve more deterministic operational coverage than generic-summary and abstract-only baselines, remain under a 1200-word compactness budget, and substantially reduce deterministic input-token proxy size relative to full extracted papers. These results support PaperToSkill as a reproducible conversion layer from curated paper notes to portable skills, while live cross-harness agent execution and human fidelity studies remain future work.


## Introduction

Using LLMs is becoming a practical skill for researchers, builders, and non-expert users. At the same time, many of the best ways to use LLM agents are introduced in papers whose methods are difficult to operationalize. A paper may describe a search policy, memory loop, validation stage, or interface contract, but a user who wants to reuse the contribution must translate prose into an agent workflow by hand.

PaperToSkill studies a simple hypothesis: papers can be converted into compact, editable skills that preserve enough procedural knowledge for agents and users to reuse the main contribution. We use "skill" to mean a natural-language operational artifact with an entry-point `SKILL.md`, similar to formats used by agent harnesses that load task-specific instructions. Unlike a summary, a skill is not only explanatory. It should specify when to use the method, which inputs are required, which steps to execute, how to validate progress, which failures to watch for, and how to adapt the instructions across harnesses.

This paper takes an artifact-first view. PaperToSkill is not presented as a fully automatic PDF understanding system. The current implementation converts curated, source-anchored paper notes into skills and source maps. This narrow scope is intentional: it lets us evaluate whether the conversion layer preserves operational knowledge before claiming end-to-end PDF automation.

Our current contributions are: (1) a paper-to-skill schema for operationalizing research contributions; (2) a deterministic extraction scaffold that emits `SKILL.md` plus source maps; (3) deterministic extracted-text-to-note scaffolds evaluated on Toolformer and AIDE; (4) a four-paper benchmark using AI Scientist-v2, Reflexion, AIDE, and Toolformer; and (5) deterministic/offline evaluations for structure, context coverage, compactness, source grounding, and transfer readiness.

## Related Work

PaperToSkill is motivated by automated research and agent workflow systems. AI Scientist-v2 uses agentic tree search and staged experiment management for automated scientific discovery (Lu et al. 2025). AIDE frames ML engineering as search over code solutions (AIDE). Reflexion stores verbal feedback in episodic memory to improve later attempts without weight updates (Shinn et al. 2023). Toolformer shows how language models can generate and filter tool-use examples for API calls (Schick et al. 2023). Skill-library and interface work such as Voyager and SWE-agent also demonstrate that reusable skills and agent-facing interfaces can determine whether a method transfers beyond its original implementation (Wang et al. 2023; Yang et al. 2024).

These papers each contribute reusable procedures, but the procedures remain inside the paper or the original system. PaperToSkill targets the conversion layer: extracting operational knowledge from papers into compact skills with validation checks, failure branches, and transfer notes.

## Method

PaperToSkill represents each converted paper as a skill package. The main file is `SKILL.md`; optional references include a source map that links generated skill bullets to source note

sections and line spans. The schema contains identity, central contribution, use conditions, required artifacts, workflow steps, validation checks, failure cases, transfer notes, and source notes that distinguish source-backed material from inferred guidance.

The deterministic extractor reads a source-anchored paper note, normalizes Markdown sections and list items, collects candidates from abstract, methods, experiments, limitations, and transfer sections, then emits a compact skill. Candidate limits keep the artifact within a practical context budget while preserving method, validation, and failure coverage. The extractor also writes a `references/source_map.json` file so later audits can inspect which source section supports each generated instruction.

The text-to-note scaffold is an earlier preprocessing step. It operates on newline-numbered extracted text, detects broad paper regions, selects line windows for methods, experiments, and limitations, and preserves line anchors. For two-column extracted text, it keeps raw line spacing and selects the keyword-bearing column while retaining the original source line range. The output is explicitly marked as an audit scaffold.

## Experimental Setup

We evaluate four real agent-method papers: AI Scientist-v2, Reflexion, AIDE, and Toolformer. For each paper, we create a curated source-anchored note from the extracted paper text, generate a PaperToSkill skill, and compare it against two context baselines: a generic summary and an abstract-only context. For transfer ablation, we compare the full skill against the same skill with `Transfer Notes` removed and against the generic summary.

The metrics are deterministic: skill rubric score, context coverage, source-span validation, offline transfer-readiness, word count under a 1200-word compactness budget, and deterministic input-token/cost proxy estimated as $\lceil \text{characters}/4 \rceil$. These metrics are reproducible gates. They do not replace live agent execution or completed human fidelity annotation. We also prepare model-ablation prompt packets for Claude Opus 4.8, a GPT-family slot requested as GPT 5.5, and a later DeepSeek slot, but those packets are recorded as pending until model aliases, responses, scores, and billing are collected.

## Results

All four generated skills score 20/20 on deterministic skill rubrics. On context coverage, the generated skills score 7.867/9 for AI Scientist-v2, 8.267/9 for Reflexion, 9.1/10 for AIDE, and 8.9/10 for Toolformer. The generic-summary and abstract-only baselines score substantially lower across all four cases (Table 1).

The generated skills remain compact: 782 words for AI Scientist-v2, 479 words for Reflexion, 927 words for AIDE, and 943 words for Toolformer, all under the 1200-word budget. Source validation finds support rates of 0.938, 1.0, 1.0, and 1.0 with zero invalid line ranges. The one weak AI Scientist-v2 span is recorded as a boundary case rather than treated as fully verified.

The context cost proxy shows the same compactness pattern relative to full paper contexts. Generated skills use between 2.2% and 9.54% of the estimated input tokens required by full extracted paper text (Table 3). We interpret this as coverage-preserving compression relative to full-paper context, not as a claim that skills are the shortest possible context.

Transfer-note ablations show a consistent offline readiness pattern. Full skills score 10/10 on the readiness metric across all four curated papers. Removing `Transfer Notes` lowers readiness to 7.6/10 in all four cases, while generic summaries score between 1.2/10 and 2.25/10. This supports the narrower claim that transfer notes encode artifact-readiness signals. It does not yet prove improved live cross-harness success.

The automatic note scaffold is evaluated separately on Toolformer and AIDE. The Toolformer auto-note-derived skill scores 20/20 on the deterministic rubric, 9.3/10 on context coverage, 10/10 on offline transfer readiness, and 1.0 source-span support with zero invalid ranges. The AIDE auto-note-derived skill scores 20/20, 8.467/10 on context coverage, 9.5/10 on offline transfer readiness, and 1.0 source-span support with zero invalid ranges (Table 4). These results support the narrower claim that extracted text can seed auditable note scaffolds; they do not establish robust arbitrary-PDF automation.

## Usage Examples and Pending Model Ablations

The repository includes usage examples for three practical workflows: loading a generated skill in a Codex-style harness, generating an automatic note scaffold from extracted text, and building model-ablation prompts. The model-ablation protocol produces a prompt grid for Claude Opus 4.8, a GPT-family slot requested as GPT 5.5, and a DeepSeek follow-up slot. The provided endpoint has previously advertised the dashed Claude alias `claude-opus-4-8`, while chat completions have been blocked by provider capacity. Therefore, the paper treats model ablations as prepared but pending rather than completed evidence.

## Discussion

The main result is not that PaperToSkill "understands" papers in the broadest sense. The result is that a structured paper-note-to-skill pipeline can preserve operational knowledge that summaries discard. The skills include workflow steps, validation checks, failure cases, and source anchors, which are exactly the parts needed when a user wants an agent to apply a paper rather than merely explain it.

The benchmark also reveals useful failure modes. During the AIDE case, the initial extractor capped workflow, validation, and failure candidates too aggressively, which dropped data-preview and LLM-cost content. Earlier phases also exposed title parsing, source-audit mapping, and source-span line-counting bugs. These records are the type of failure branches PaperToSkill should preserve: not just final successes, but the ways extraction, evaluation, or external dependencies can lose important operational content.

Table 1: Main deterministic/offline PaperToSkill results. Coverage scores are task-rubric scores; transfer readiness is an offline artifact-readiness metric.

| Paper | Rubric | Skill | Summary | Abstract | ΔSum. | Transfer | Support | Words |
|---|---|---|---|---|---|---|---|---|
| AI Scientist-v2 | 20/20 | 7.867/9 | 1.733/9 | 1.2/9 | 6.134 | 10/10 | 0.938 | 782 |
| Reflexion | 20/20 | 8.267/9 | 3.483/9 | 2.533/9 | 4.784 | 10/10 | 1.000 | 479 |
| AIDE | 20/20 | 9.1/10 | 1.916/10 | 1.333/10 | 7.184 | 10/10 | 1.000 | 927 |
| Toolformer | 20/20 | 8.9/10 | 2.5/10 | 1.534/10 | 6.400 | 10/10 | 1.000 | 943 |

Table 2: Transfer-note ablation. Removing only `Transfer Notes` lowers offline readiness across all curated skills.

| Paper | Full | No Transfer |
|---|---|---|
| AI Scientist-v2 | 10/10 | 7.6/10 |
| Reflexion | 10/10 | 7.6/10 |
| AIDE | 10/10 | 7.6/10 |
| Toolformer | 10/10 | 7.6/10 |

## Limitations

The current work has several important limits. First, the main benchmark converts curated notes rather than arbitrary PDFs; automatic note scaffolds have only been retained on Toolformer and AIDE extracted text. Second, the metrics are deterministic and partly lexical, so they can over-credit exact matches or under-credit valid paraphrases. Third, live cross-harness execution has not completed because the provided remote endpoint returned service errors during chat-completion tests. Fourth, human-fidelity packets, a blank annotation template, and a summary script are prepared, but no independent annotations have been completed. Fifth, compactness is measured by word count and deterministic input-token proxy, not by tokenizer-exact model price, provider billing, or live success per dollar.

## Conclusion

PaperToSkill turns paper-derived procedural knowledge into portable, human-editable skills. In a four-paper benchmark, generated skills are compact, source-grounded, structurally valid, and more operationally complete than short summary baselines under deterministic evaluation. The next stage is to execute the prepared live prompt packets across agent harnesses and model families, add human fidelity review, compute tokenizer-exact and provider-billing costs, and test papers whose contributions are less naturally procedural.

Table 3: Deterministic context-size proxy. Tokens are estimated as $\lceil \text{characters}/4 \rceil$ and are not provider bills or tokenizer-exact counts.

| Paper | Full Paper Tokens | Skill Tokens | Skill/Full | Reduction |
|---|---|---|---|---|
| AI Scientist-v2 | 62041 | 1366 | 0.0220 | 97.80% |
| Reflexion | 18559 | 823 | 0.0443 | 95.57% |
| AIDE | 15894 | 1517 | 0.0954 | 90.46% |
| Toolformer | 24097 | 1526 | 0.0633 | 93.67% |

Table 4: Automatic extracted-text note scaffolds are evaluated separately from curated-note cases.

| Paper | Input | Rubric | Coverage | Transfer | Support | Words |
|---|---|---|---|---|---|---|
| Toolformer | Curated note | 20/20 | 8.9/10 | 10/10 | 1.000 | 943 |
| Toolformer | Auto note scaffold | 20/20 | 9.3/10 | 10/10 | 1.000 | 1179 |
| AIDE | Curated note | 20/20 | 9.1/10 | 10/10 | 1.000 | 927 |
| AIDE | Auto note scaffold | 20/20 | 8.467/10 | 9.5/10 | 1.000 | 998 |

## References


AIDE. 2025. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.